OpenZeppelin | security

Contango Core v3 Audit



April 10, 2026

This document has been prepared solely for the named client and it may not be relied upon by any third parties. The content in this document is provided for informational purposes only and does not constitute professional advice of any kind. Security assessments are limited by factors such as scope, time, and techniques used and therefore this document does not constitute a certification, guarantee, or warranty regarding the proper functioning of any system or the absence of security vulnerabilities.

Copyright © 2026 Zeppelin Group Ltd.

Table of Contents

Table of Contents 2

Summary 4

Scope 5

System Overview 6

Executor Modules 6

Action Execution 6

Validator Modules 7

NFT Callback Handler 7

Security Model and Trust Assumptions 7

Medium Severity 8

M-01 Payable Delegate and Action Entry Points Do Not Forward ETH to the Account 8

Low Severity 9

L-01 Missing NatSpec Documentation 9

L-02 Unchecked Return Values 9

L-03 isModuleType Always Returns false 10

L-04 Batch Execution Results Omit Success Status for Allowed-Failure Actions 10

L-05 Inconsistent Validation in transferOut Overloads 11

L-06 Owner Registration Without Module Installation 11

L-07 False OwnerRemoved Emission for Non-Owner Addresses 12

L-08 Base Module onInstall and onUninstall Do Not Enforce Installation State 12

L-09 Incomplete ERC-165 Implementation in NFT Callback Handler 13

L-10 Floating Pragma Can Lead to Inconsistent Compilation 13

L-11 Unconditional Collateral Check Makes Loan Token Withdrawals Unreachable 14

L-12 Missing Entrypoint Discriminator in EIP-712 Signature Allows Theoretical Cross-Entrypoint Replay 14

Notes & Additional Information 15

N-01 Redundant Event Emission in Single Swap Execution 15

N-02 Redundant Data Length Check in Module Installation 16

N-03 Hash Approval Does Not Prevent Redundant Signatures 16

N-04 Source Reference Links to a Mutable Branch 16

N-05 Uncached Array Length in Validation Loop 17

N-06 Implicit Return in Unimplemented Stateless Validation 17

N-07 Function Visibility Overly Permissive 17

N-08 Missing Indexed Parameters in Ownership Change Events 18

N-09 Missing Security Contact 18

N-10 Unused Internal Function for Nonce Status Checks 19

N-11 Redundant Getter Function 19
N-12 flashBorrowMany Reverts for Multi-Asset Borrows 19

Conclusion 21

Appendix 22
Issue Classification 22

Summary

Type	DeFi
Timeline	From 2026-03-23 To 2026-04-03
Languages	Solidity

Total Issues	25 (23 resolved)
Critical Severity Issues	0 (0 resolved)
High Severity Issues	0 (0 resolved)
Medium Severity Issues	1 (1 resolved)
Low Severity Issues	12 (11 resolved)
Notes & Additional Information	12 (11 resolved)
Client Reported Issues	0 (0 resolved)

Scope

OpenZeppelin performed an audit of the contango-xyz/core-v3 repository at the 96fffe7 commit.

In scope were the following files:

```
src
├── constants.sol
├── modules
│   ├── ActionExecutor.sol
│   ├── ERC1271Executor.sol
│   ├── MultiValidator.sol
│   ├── NFTCallbackHandler.sol
│   ├── OwnableExecutor.sol
│   ├── PreSignedValidator.sol
│   ├── UnorderedNonce.sol
│   └── base
│       ├── ERC7579Executor.sol
│       ├── ERC7579Hook.sol
│       ├── ERC7579Module.sol
│       ├── ERC7579StatelessValidator.sol
│       └── ERC7579Utils.sol
└── types
    └── Action.sol
```

System Overview

The audited codebase implements a modular ERC-7579 module suite for smart contract accounts. It provides executor modules that authorize and forward operations to the account, validator modules that verify signatures and hash approvals, and supporting infrastructure for action execution and NFT callbacks. The modules are designed to be installed on any ERC-7579-compliant smart account and operate as external contracts that interact with the account through the standardized `executeFromExecutor` interface.

Executor Modules

The executor layer is responsible for authorizing operations and forwarding them to the smart account for execution. Two executor implementations are provided. `OwnableExecutor` gates operations behind an owner whitelist managed through an `EnumerableSet`, supporting single calls, batch calls, delegate calls, and action execution. `ERC1271Executor` gates operations behind EIP-712 typed signatures validated through the account's ERC-1271 interface, combined with an unordered nonce scheme derived from Uniswap's Permit2 design. Both executors inherit from `ERC7579Executor`, which encodes ERC-7579 execution modes, manages ETH forwarding to the account, and routes action execution through a shared `ActionExecutor` via delegatecall.

Action Execution

`ActionExecutor` serves as a dispatch layer between executor modules and stateless action contracts. It unpacks tightly-encoded `PackedAction` structs, containing target address, value, delegate call flag, allow-failure flag, and calldata in a gas-efficient byte layout and executes them as either regular calls or delegate calls in the account's context. The action system supports both single and batch execution, with per-action failure tolerance controlled by the allow-failure flag.

Validator Modules

Two validator modules handle signature and hash approval logic for ERC-4337 UserOps and ERC-1271 signature checks. `PreSignedValidator` supports both permanent and ephemeral hash approvals, allowing accounts to pre-authorize specific digests. `MultiValidator` enables batch intent signing, where a signer authorizes up to ten operation hashes in a single signature that can be validated independently.

NFT Callback Handler

`NFTCallbackHandler` is a fallback module that implements the ERC-721 and ERC-1155 receiver interfaces, allowing smart accounts to accept NFT transfers. It optionally executes an embedded call from the transfer data when present.

Security Model and Trust Assumptions

The security model relies on the ERC-7579 module installation mechanism as the root of trust. Modules are installed and uninstalled by the account itself, and all execution flows originate from either an authenticated owner or a validated signature. Once authorized, the full action set is available with no per-action access control.

Medium Severity

M-01 Payable Delegate and Action Entry Points Do Not Forward ETH to the Account

The base `ERC7579Executor` provides a `_forwardValue` helper that transfers `msg.value` to the account before calling `executeFromExecutor`, since some account implementations (e.g. Safe7579) have a non-payable executor entrypoint. This helper is called in the `_execute` and `_executeBatch` paths but not in `_delegate`.

Since `delegate`, `executeAction`, and `executeActions` on both `OwnableExecutor` and `ERC1271Executor` are marked `payable` and all route through `_delegate`, any ETH sent with these calls remains on the executor contract. The failure flow is as follows:

1. A caller invokes `executeAction{value: X}(account, action)` on the executor module.
2. The call reaches `_delegate`, which calls `account.executeFromExecutor` without first sending the ETH.
3. The account delegatecalls into `ActionExecutor`, which attempts to spend from `address(this).balance` (the account's own balance).
4. The caller's ETH stays trapped on the executor contract with no recovery mechanism.

This results in either silent loss of the caller's ETH or an `InsufficientBalance` revert if the account does not already hold sufficient funds.

Consider forwarding `msg.value` inside `_delegate` by calling `_forwardValue(address(account))`, matching the behavior of `_execute` and `_executeBatch`. Alternatively, consider making these entrypoints non-payable if value-funded delegatecalls are not intended.

Update: *Resolved in pull request #3 at commit dd54626.*

Low Severity

L-01 Missing NatSpec Documentation

Multiple contracts and files in the codebase lack NatSpec docstrings. This includes, for example, the public flash loan entry points for the supported protocols (`ERC7399`, `ERC3156`, `Aave`, `Balancer`, `Morpho`, `Uniswap V3`, `Algebra`, `Solidly`, `Euler`, and `Pendle`), as well as callback implementations and internal helper functions. The absence of NatSpec comments makes the codebase harder to understand, review, and integrate correctly.

Consider adding NatSpec documentation throughout the codebase, with priority given to external and public entry points, callback functions, and core internal helpers.

Update: *Resolved in pull request #3 at commits 3de93a6, 86ae81c, and 024bee2.*

L-02 Unchecked Return Values

Several functions discard return values from helpers that indicate whether an operation had any effect. As a result, no-op operations can proceed and events can suggest that state changes occurred when they did not.

In `OwnableExecutor`, `_addOwner` and `_removeOwner` ignore the boolean returned by `EnumerableSet.add` and `EnumerableSet.remove`. These functions return `false` when an owner already exists or does not exist, respectively. Because the result is not checked, `OwnerAdded` can be emitted for duplicate additions and `OwnerRemoved` can be emitted for addresses that were never owners. The set state remains correct, but off-chain consumers of these events may infer an incorrect ownership history.

In `FlashLoanAction`, `flashLoanERC3156` calls `provider.flashLoan()` without checking its return value. ERC-3156 requires the lender to return `true` on success. Without this validation, a non-compliant provider could fail silently without reverting.

In `ERC20Lib`, `depositNative` and `transferOutNative` call `_validAmount(amount)` but ignore its boolean return value. Since `_validAmount` returns `false` when `amount == 0`, both functions still execute on zero amounts, waste gas, and emit misleading `NativeTokenDeposited` or `NativeTokenWithdrawn` events. This differs from the ERC-20 `transferOut` overload, which checks the return value and exits early.

Consider checking each return value before proceeding: return early for zero-amount native token operations, emit ownership events only when the underlying set mutation succeeds, and require `provider.flashLoan()` to return `true`.

Update: *Resolved in pull request #3 at commit 104d2da.*

L-03 `isModuleType` Always Returns `false`

In `ERC7579StatelessValidator.sol`, `isModuleType` checks the provided module type ID against both `MODULE_TYPE_VALIDATOR` and `MODULE_TYPE_STATELESS_VALIDATOR` using `&&`. Because a single value cannot equal both constants at the same time, the condition always evaluates to `false`.

This causes the base implementation to report that it supports no module type. The practical impact may be limited because `isModuleType` is overridden in `MultiValidator` and `PreSignedValidator`.

Consider replacing `&&` with `||` so the function returns `true` when the provided type matches either supported constant.

Update: *Resolved in pull request #3 at commit caee16d.*

L-04 Batch Execution Results Omit Success Status for Allowed-Failure Actions

In `ActionExecutor`, the `executeSingle` function supports an `allowFailure` flag for actions. When an action with `allowFailure` set to `true` reverts, execution continues and the revert is not propagated. However, the function returns only the raw `bytes result`, with no indication of whether the action succeeded or failed.

This behavior carries through to `executeBatch` and `executeBatchPacked`, which return `bytes[]` to the caller. As a result, callers cannot distinguish a successful return value from return data produced by a failed allowed-failure action, because both are placed in the same `bytes` slot. Any module or integration that uses batch results for downstream decisions, such as decoding return data, could misinterpret failure output as a valid result.

Consider returning each batch result together with an explicit success flag, or otherwise encoding the success status so callers can reliably identify which actions failed.

Update: *Resolved in pull request #3 at commit 07b328d.*

L-05 Inconsistent Validation in `transferOut` Overloads

`ERC20Lib` exposes multiple `transferOut` overloads, but they do not enforce the same preconditions. The base overload validates the amount, payer, and destination before proceeding. The permit-based overload repeats these checks.

The `usePermit2` overload does not perform these validations. When `usePermit2` is `true`, it calls `PERMIT2.transferFrom` without checking for a zero amount, zero payer, or zero destination. When `usePermit2` is `false`, it delegates to the base overload, where those checks do run. As a result, callers receive different validation guarantees depending on which overload or branch is used, which can lead to inconsistent behavior and make the API easier to misuse.

Consider centralizing input validation so all `transferOut` entry points enforce the same guards, or clearly documenting which overloads are expected to validate inputs and which rely on callers to do so.

Update: *Resolved in pull request #3 at commit 7578db1.*

L-06 Owner Registration Without Module Installation

The single-argument `addOwner` function allows any smart account (as `msg.sender`) to register an owner without having the module installed. This creates a problematic state:

1. An account calls `addOwner(someAddress)`, populating the owner set.
2. The account later attempts to install the module via `onInstall`, which requires `accountOwners[account].length() == 0` and reverts with `AlreadyInstalled`.
3. The account must first call `removeOwner` for each pre-registered address before installation can proceed.

While recovery is possible through `removeOwner` (which is also `msg.sender`-gated), the intermediate state is confusing and the `AlreadyInstalled` error is misleading since the module was never actually installed.

Consider adding an installation check to the single-argument `addOwner`, or allowing `onInstall` to succeed when owners were pre-registered by the same account.

Update: *Resolved in pull request #3 at commit 602cb2d.*

L-07 False `OwnerRemoved` Emission for Non-Owner Addresses

The single-argument `removeOwner` function allows the caller to remove any address from its owner set. However, `_removeOwner` does not check the return value of `EnumerableSet.remove`. As a result, calling `removeOwner` with an address that is not currently an owner emits an `OwnerRemoved` event even though no state change occurs.

Access remains correctly restricted to `msg.sender`, but repeated calls with arbitrary non-owner addresses can pollute logs and mislead off-chain systems that infer ownership from events.

Consider reverting when the target address is not a current owner, or otherwise only emitting `OwnerRemoved` when `EnumerableSet.remove` returns `true`.

Update: *Resolved in pull request #3 at commit 104d2da.*

L-08 Base Module `onInstall` and `onUninstall` Do Not Enforce Installation State

The default `onInstall` and `onUninstall` implementations in `ERC7579Module` only emit events without tracking or enforcing installation state. According to ERC-7579, `onInstall` MUST revert if the module is already enabled and `onUninstall` MUST revert on error. The base contract satisfies neither requirement.

While `OwnableExecutor` overrides both hooks with proper guards, other concrete modules such as `NFTCallbackHandler` rely on the unguarded defaults.

Consider tracking installation state in the base contract and reverting on duplicate installs and on uninstalls of non-installed modules by default.

Update: *Resolved in pull request #3 at commit 8dc0e6c.*

L-09 Incomplete ERC-165 Implementation in NFT Callback Handler

The `supportsInterface` function in `NFTCallbackHandler` returns `true` for `IERC721Receiver` and `IERC1155Receiver`, but not for the ERC-165 interface identifier `0x01ffc9e7`. Under ERC-165, a contract that implements `supportsInterface` is expected to report support for `IERC165` itself.

As a result, external contracts or registries that rely on standard ERC-165 introspection may treat `NFTCallbackHandler` as non-compliant and skip it during interface discovery.

Consider updating `supportsInterface` to return `true` for `type(IERC165).interfaceId`, or inheriting from OpenZeppelin's `ERC165` base contract if that matches the intended design.

Update: *Resolved in pull request #3 at commit a2e3640.*

L-10 Floating Pragma Can Lead to Inconsistent Compilation

All contracts in the codebase use floating pragma directives such as `^0.8.20` or `^0.8.0`. Although `foundry.toml` pins the compiler to Solidity `0.8.34`, the source files do not enforce that version themselves. If the contracts are compiled outside the Foundry configuration, such as by integrators importing them, a different compiler version may be selected.

This can lead to inconsistent compilation behavior across environments and may introduce version-dependent differences.

Consider using a fixed pragma directive to enforce consistent compilation across environments.

Update: *Acknowledged, not resolved. The Contango team stated:*

> We use the foundry versioning by design and we don't expect integrators to be building this code, only coding to our interfaces.

L-11 Unconditional Collateral Check Makes Loan Token Withdrawals Unreachable

The `withdraw` function in `MorphoMoneyMarket` is intended to support withdrawals of both the collateral token and the loan token.

However, it unconditionally calls `collateralBalance` before branching on the token type. Because `collateralBalance` requires the provided token to equal the market's collateral token, any `withdraw` call that passes the loan token reverts with `InvalidToken` before execution can reach the loan-token branch. As a result, loan token withdrawals through this adapter are unreachable.

Consider moving the `collateralBalance` call into the collateral-token branch, or otherwise gating it so it only executes when `amount == COLLATERAL_BALANCE` and `token` is the collateral token. This preserves full-balance collateral withdrawals without blocking the loan-token path.

Update: *Resolved in pull request #3 at commits 94786ed and 4887cae.*

L-12 Missing Entrypoint Discriminator in EIP-712 Signature Allows Theoretical Cross-Entrypoint Replay

The `ERC1271Executor` gates multiple execution entrypoints (`execute`, `delegate`, `executeAction`, `executeActions`, `executeBatch`) behind EIP-712 signatures using a shared `EXECUTION_TYPEHASH`. The signed struct contains `account`, `accountData`, and `nonce`, but no field identifying which entrypoint the signature is intended for.

This creates a byte-level collision between `executeAction` and `delegate`. The `executeAction` entrypoint signs over raw `action.data`, while `delegate` signs over `encodeDelegate(target, data)` which produces `abi.encodePacked(target, data)`. Since `PackedAction.data` starts with a 20-byte target address followed by value and flags, an attacker observing a signature intended for `executeAction` could submit it to `delegate` with `target = action.data[:20]` and `data = action.data[20:]`, producing an identical digest.

In practice, this collision does not lead to exploitable harm. The replayed `delegate` call would delegatecall into the original action target with garbled calldata (the uint96 value field becomes the function selector prefix). For any practical value amount, the first four bytes are

zeros, producing an invalid selector that causes the call to revert. Since the entire transaction reverts, the nonce in the shared `UnorderedNonce` bitmap is not consumed, and the originally intended operation remains executable. Similar collision pairs exist between `execute` and `delegate` with the same outcome.

Consider adding an entrypoint discriminator (such as the function selector or an execution mode identifier) to the `EXECUTION_TYPEHASH` struct to bind each signature to its intended entrypoint and eliminate the collision as a defense-in-depth measure.

Update: *Resolved in pull request #3 at commit b0a9da3.*

Notes & Additional Information

N-01 Redundant Event Emission in Single Swap Execution

The `executeSwap` function calls `_executeSwap`, which emits a `SwapPartExecuted` event, and then emits a `SwapExecuted` event itself. For a single swap, both events carry the same information (token pair, amounts), making the `SwapPartExecuted` emission redundant. The `SwapPartExecuted` event is designed for the multi-swap path in `executeSwaps`, where it provides per-step granularity that `SwapExecuted` summarizes at the end.

Consider suppressing the `SwapPartExecuted` event when the swap is invoked through the single-swap entry point, or documenting that consumers should expect both events for single swaps.

Update: *Acknowledged, not resolved. The Contango team stated:*

> *Won't add the extra complexity here because it will also add extra complexity for indexers.*

N-02 Redundant Data Length Check in Module Installation

In `OwnableExecutor.onInstall`, the check `data.length >= ADDR_SIZE` is redundant when `data.length % ADDR_SIZE == 0` is already enforced. If `data.length` is zero, the modulo check still passes, and the decoded owner count becomes zero. That case can be handled directly by validating the computed number of addresses instead of the raw byte length.

Consider removing the `data.length >= ADDR_SIZE` requirement and validating that the decoded address count is greater than zero after the modulo check, rather than relying on the separate `AtLeastOneOwner` guard.

Update: *Resolved in pull request #3 at commit fc40abb.*

N-03 Hash Approval Does Not Prevent Redundant Signatures

The `_sign` function in `PreSignedValidator` unconditionally writes the signing state without checking whether the hash is already signed. Calling `approveHash` on an already-approved hash overwrites true with true and emits a duplicate `HashSigned` event. The same applies to `revokeHash` on an already-revoked hash. While the on-chain state remains correct due to idempotency, the duplicate events can mislead off-chain systems tracking signature status.

Consider checking the current signing state before writing and emitting, or documenting that duplicate approvals are expected and harmless.

Update: *Resolved in pull request #3 at commit 104d2da.*

N-04 Source Reference Links to a Mutable Branch

The source attribution comment in `UnorderedNonce` points to the Permit2 repository's `main` branch. Because `main` can change over time, the link may eventually reference different code than the version that was originally adapted, which reduces its value as a stable implementation reference.

Consider replacing the branch-based reference with a permalink to the specific upstream commit that was used.

Update: *Resolved in pull request #3 at commit 1f51b9a.*

N-05 Uncached Array Length in Validation Loop

The `_validate` function in `MultiValidator` caches `multiSignature.intents.length` into a local variable but then reads the array length directly in the loop condition. This results in a redundant memory read on each iteration instead of using the already-cached stack variable.

Consider replacing `multiSignature.intents.length` in the loop condition with the existing `length` variable.

Update: *Resolved in pull request #3 at commit 0eae37e.*

N-06 Implicit Return in Unimplemented Stateless Validation

The `MultiValidator` contract does not support stateless validation. However, `validateSignatureWithData` has an empty body and therefore relies on Solidity's implicit default return value of `false`. Although this is functionally correct, the lack of an explicit return makes the intent unclear and may be mistaken for an incomplete implementation.

Consider returning `false` explicitly in `validateSignatureWithData` to make the unsupported behavior unambiguous.

Update: *Resolved in pull request #3 at commit f37d4d1.*

N-07 Function Visibility Overly Permissive

Multiple functions across the codebase have visibility broader than their current usage requires. Several `public` functions are never called internally (and could be `external`), and several `internal` functions are only used within the declaring contract (and could be `private`). Notable cases include `internal` helpers in `ERC7579Executor` (`_call`, `_delegate`, `_batch`, `_forwardValue`), library functions in `ERC7579Utils`

(`encodeSingle`, `encodeDelegate`, `encodeBatch`), and various `internal` helpers in `OwnableExecutor` and `PreSignedValidator`.

However, many of these are in abstract base contracts or libraries intentionally designed for inheritance and extension. Restricting `internal` to `private` would prevent subclasses from overriding or reusing them. For library functions, `internal` is the correct visibility since libraries cannot have `private` functions callable by importing contracts.

Consider tightening visibility only for functions in non-abstract, non-library contracts where no inheritance-based extension is anticipated. Changing from `public` to `external` is safe and can potentially provide minor gas savings on calldata-heavy functions.

Update: *Resolved in pull request #3 at commits 1304f2c and 86e5cf5.*

N-08 Missing Indexed Parameters in Ownership Change Events

The `OwnerAdded` and `OwnerRemoved` events in `OwnableExecutorEvents` include `IERC7579Execution account` and `address owner` parameters, but neither parameter is indexed. As a result, off-chain services that need to track ownership changes for a specific account or owner cannot use topic-based filtering and must instead scan and decode all matching logs.

Consider indexing relevant event parameters to ease off-chain indexing.

Update: *Resolved in pull request #3 at commit 922f9be.*

N-09 Missing Security Contact

No contract in the codebase includes a `@custom:security-contact` NatSpec annotation. This annotation is commonly used to provide a clear channel for responsible vulnerability disclosure. Given that these contracts are intended to be installed on user smart accounts that manage real funds, the absence of a security contact may delay the reporting of security issues.

Consider adding a `@custom:security-contact` NatSpec annotation to each contract definition that points to Contango's security disclosure channel.

Update: *Resolved in pull request #3 at commit 723398e.*

N-10 Unused Internal Function for Nonce Status Checks

The `_wasNonceUsed` function in `UnorderedNonce` is not called anywhere in the current codebase. The function provides a read-only check for whether a nonce has been consumed, but it currently has no on-chain consumer.

Consider using the `_wasNonceUsed` function in `_useUnorderedNonce` to check whether a nonce is taken. Otherwise, consider removing the function if no current or planned usage exists.

Update: *Resolved in pull request #3 at commit 8f3d45d.*

N-11 Redundant Getter Function

In `ERC7579Executor`, the `actionExecutor` function manually returns the public immutable `ACTION_EXECUTOR`, which already has an auto-generated getter. This creates two accessors for the same value: `actionExecutor()` and `ACTION_EXECUTOR()`.

This may be intentional for interface compatibility, but if not, the duplicate accessor adds unnecessary surface area and may cause minor confusion for integrators.

Consider removing the redundant getter if `ACTION_EXECUTOR()` is sufficient, or documenting why both accessors are required if `actionExecutor()` must be preserved for external compatibility.

Update: *Resolved in pull request #3 at commit a107052.*

N-12 `flashBorrowMany` Reverts for Multi-Asset Borrows

In `AaveMoneyMarket`, the `flashBorrowMany` function builds `interestRateModes` by calling `uint256s()` on a single value. This always returns a one-element array, regardless of how many assets are requested. If `assets` contains more than one token, Aave's pool reads past the end of `interestRateModes` when processing the second asset, and the call reverts.

This means `flashBorrowMany` only works for single-asset borrows, despite its interface suggesting support for multiple assets.

Consider either constructing `interestRateModes` with the same length as `assets` or, if multi-asset flash borrowing is not intended for this adapter, documenting that limitation and rejecting calls with more than one asset.

Update: *Resolved in pull request #3 at commit 465812a.*

Conclusion

The audited codebase implements a modular ERC-7579 smart account framework for composable DeFi execution, covering the module authorization layer, stateless action primitives, and supporting infrastructure.

No high- or critical-severity issues were identified. One medium severity finding was raised regarding ETH value handling in delegatecall-based execution paths, where payable entrypoints fail to forward `msg.value` to the account. The remaining findings were low severity or informational, addressing topics such as missing installation state tracking in the base module, incomplete ERC-165 support, and inconsistent input validation across library overloads.

The codebase is well-structured with a clean separation between authorization modules and execution actions. The codebase would benefit from more comprehensive NatSpec documentation and expanded test coverage, particularly for value-forwarding paths in the executor layer, where no tests currently exercise `msg.value` through the delegatecall-based entrypoints.

We would like to thank the Contango team for being responsive throughout the engagement and providing detailed context on the system's design rationale.

Appendix

Issue Classification

OpenZeppelin classifies smart contract vulnerabilities on a 5-level scale:

- Critical
- High
- Medium
- Low
- Note/Information

Critical Severity

This classification is applied when the issue's impact is catastrophic, threatening extensive damage to the client's reputation and/or causing severe financial loss to the client or users. The likelihood of exploitation can be high, warranting a swift response. Critical issues typically involve significant risks such as the permanent loss or locking of a large volume of users' sensitive assets or the failure of core system functionalities without viable mitigations. These issues demand immediate attention due to their potential to compromise system integrity or user trust significantly.

High Severity

These issues are characterized by the potential to substantially impact the client's reputation and/or result in considerable financial losses. The likelihood of exploitation is significant, warranting a swift response. Such issues might include temporary loss or locking of a significant number of users' sensitive assets or disruptions to critical system functionalities, albeit with potential, yet limited, mitigations available. The emphasis is on the significant but not always catastrophic effects on system operation or asset security, necessitating prompt and effective remediation.

Medium Severity

Issues classified as being of medium severity can lead to a noticeable negative impact on the client's reputation and/or moderate financial losses. Such issues, if left unattended, have a moderate likelihood of being exploited or may cause unwanted side effects in the system.

These issues are typically confined to a smaller subset of users' sensitive assets or might involve deviations from the specified system design that, while not directly financial in nature, compromise system integrity or user experience. The focus here is on issues that pose a real but contained risk, warranting timely attention to prevent escalation.

Low Severity

Low-severity issues are those that have a low impact on the client's operations and/or reputation. These issues may represent minor risks or inefficiencies to the client's specific business model. They are identified as areas for improvement that, while not urgent, could enhance the security and quality of the codebase if addressed.

Notes & Additional Information Severity

This category is reserved for issues that, despite having a minimal impact, are still important to resolve. Addressing these issues contributes to the overall security posture and code quality improvement but does not require immediate action. It reflects a commitment to maintaining high standards and continuous improvement, even in areas that do not pose immediate risks.